

BIG DATA ANALYTICS

Assignment-1

Name: Subhapreet Patro

Roll No.: 2211CS010547

Section: I

Q1) Write Short notes: a). Data Integrity b). Compression

A)

a) Data Integrity in Big Data Analytics

Definition:

Data integrity refers to the **accuracy, consistency, and reliability** of data throughout its lifecycle. In Big Data Analytics, maintaining data integrity ensures that the data being collected, processed, and analyzed is **correct, trustworthy, and usable**.

Key Aspects in Big Data:

- **Volume and Velocity:** Due to massive and fast data inflow, ensuring integrity becomes challenging.
- **Data Validation:** Implementing rules and constraints to verify correctness.
- **ETL (Extract, Transform, Load) Pipelines:** Must preserve the structure and meaning of data during transformation.
- **Checksums and Hashing:** Used for verifying data consistency during transfer or storage.
- **Audit Trails:** Keeping logs of changes helps track data alterations and ensure traceability.

Importance:

- Poor data integrity can lead to **misleading analytics** and **faulty decision-making**.
- Essential for **regulatory compliance** (e.g., GDPR, HIPAA).
- Boosts **trust** in analytics outcomes among stakeholders.

b) Compression in Big Data Analytics

Definition:

Compression refers to techniques used to **reduce the size of data** stored or transmitted. In Big Data Analytics, it helps manage **storage costs, improve processing speed, and reduce data transfer time**.

Types of Compression:

- **Lossless Compression:** No data is lost; essential for analytics (e.g., Snappy, GZIP, BZIP2).
- **Lossy Compression:** Some data is lost; suitable for media files (e.g., audio, images).

Use Cases in Big Data:

- **Hadoop Distributed File System (HDFS):** Uses compression to store huge volumes efficiently.
- **Apache Parquet / ORC formats:** Columnar storage formats that support efficient compression and are optimized for query performance.
- **Streaming Data:** Real-time systems (e.g., Kafka) use compression to reduce network load.

Benefits:

- **Reduced Storage Needs:** Saves cost and allows more data to be retained.
- **Faster Data Transfer:** Improves I/O performance during data movement.
- **Efficient Query Processing:** Smaller data size improves analytics engine speed.

Q2) What is MapReduce? Explain key components of Map Reduce

A)

What is MapReduce?

MapReduce is a **programming model and processing framework** developed by **Google** to process and generate large datasets in a **parallel, distributed** manner across clusters of computers.

It is the core component of **Hadoop** and is designed to handle **big data processing** by breaking tasks into smaller sub-tasks that can be executed in parallel.

Main Idea:

MapReduce works in two primary phases — **Map** and **Reduce** — where:

- The **Map phase** processes input data and converts it into intermediate key-value pairs.
- The **Reduce phase** aggregates, summarizes, or transforms those intermediate results to generate the final output.

Key Components of MapReduce

◆ 1. Map Function

- **Input:** Raw data in (key, value) format.
- **Process:** Performs operations like filtering, sorting, parsing, etc.
- **Output:** Generates intermediate (key, value) pairs.

Example:

If input is a text document, the Map function could emit a key-value pair like:

Map("file.txt", "Hadoop is fast") → ("Hadoop", 1), ("is", 1), ("fast", 1)

◆ 2. Shuffle and Sort

- Occurs **after the Map phase and before Reduce phase**.
- Groups all intermediate values by key and sorts them.
- Ensures that all values belonging to the same key are sent to the same reducer.

Purpose:

- Prepares the data for effective reduction.
- Example:

Before Reduce: [("Hadoop", 1), ("Hadoop", 1), ("is", 1)]

After Shuffle & Sort: ("Hadoop", [1, 1]), ("is", [1])

◆ 3. Reduce Function

- **Input:** A key and a list of values associated with it.
- **Process:** Aggregates the values (e.g., sums, averages, max, etc.)
- **Output:** A final (key, value) pair.

Example:

Reduce("Hadoop", [1, 1]) → ("Hadoop", 2)

◆ 4. JobTracker (Master - in Hadoop v1)

- Coordinates all the MapReduce jobs.
- Assigns tasks to various nodes (TaskTrackers).
- Monitors the progress and handles failures.

In Hadoop 2.x (YARN), **JobTracker** is replaced by **ResourceManager** and **ApplicationMaster**.

◆ **5. TaskTracker (Worker Node - in Hadoop v1)**

- Executes the tasks assigned by the JobTracker.
- Periodically sends heartbeat signals to confirm it's alive.

In Hadoop 2.x, replaced by **NodeManager**.

◆ **6. InputFormat and OutputFormat**

- **InputFormat:** Defines how input files are split and read.
 - E.g., TextInputFormat, KeyValueTextInputFormat
 - **OutputFormat:** Defines how the output is written.
 - E.g., TextOutputFormat, SequenceFileOutputFormat
-

Working Flow of MapReduce

Step 1: Input data → InputFormat → Split into blocks

Step 2: Each block → Map function → (key, value) pairs

Step 3: Intermediate data → Shuffle & Sort → Grouped by key

Step 4: Reduce function → Final (key, value) output

Step 5: OutputFormat → Write to HDFS or local FS

Example: Word Count Using MapReduce

Input:

file1.txt → "big data big problems"

Map Output:

("big", 1), ("data", 1), ("big", 1), ("problems", 1)

Shuffle and Sort:

("big", [1, 1]), ("data", [1]), ("problems", [1])

Reduce Output:

("big", 2), ("data", 1), ("problems", 1)

Advantages of MapReduce

- Scalable to petabytes of data
- Fault-tolerant
- Parallel and distributed processing
- Cost-effective on commodity hardware

Limitations

- Not suitable for real-time processing
- Programming model is low-level and complex
- Inefficient for iterative tasks (like ML)

Q3) What is Hadoop Streaming? Differentiate Hadoop Streaming and Hadoop Pipes

A)

What is Hadoop Streaming?

Hadoop Streaming is a **utility** that comes with the Hadoop distribution and allows users to **create MapReduce jobs using any scripting or programming language** that can read from standard input (stdin) and write to standard output (stdout).

It provides flexibility by **not limiting developers to Java**, which is the default language for writing MapReduce programs in Hadoop.

◆ How Hadoop Streaming Works:

1. **Input data** is passed line-by-line to the **mapper** script via stdin.
2. The **mapper script** processes the data and writes output key-value pairs to stdout.
3. These intermediate key-value pairs go through **shuffle and sort**.
4. The **reducer script** takes grouped key-value pairs via stdin and writes final output to stdout.

◆ Example: Using Python Scripts

Mapper (mapper.py):

```
#!/usr/bin/env python3  
  
import sys
```

```
for line in sys.stdin:
    for word in line.strip().split():
        print(f"{word}\t1")
```

Reducer (reducer.py):

```
#!/usr/bin/env python3
import sys
from collections import defaultdict

word_count = defaultdict(int)

for line in sys.stdin:
    word, count = line.strip().split('\t')
    word_count[word] += int(count)
```

```
for word in word_count:
    print(f"{word}\t{word_count[word]}")
```

Run the Job with:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
    -input /input/data \
    -output /output/result \
    -mapper mapper.py \
    -reducer reducer.py
```

What are Hadoop Pipes?

Hadoop Pipes is a **C++ API** for writing MapReduce programs that run within the Hadoop framework. It allows developers to write **high-performance, native C++ MapReduce applications** that can be executed on Hadoop clusters.

It uses **Java Native Interface (JNI)** to connect the C++ application to the Java-based Hadoop system.

◆ Key Features of Hadoop Pipes:

- Enables tight integration with **native C++ applications**.
- Provides **object-oriented** interfaces to implement mapper, reducer, and combiner.
- Offers **better performance** than streaming due to native execution and lower overhead.

Difference between Hadoop Streaming and Hadoop Pipes

Feature	Hadoop Streaming	Hadoop Pipes
Language Support	Any language (Python, Perl, Ruby, etc.)	C++ only
Interface Type	Uses standard input/output (stdin, stdout)	Uses JNI to connect C++ code with Java Hadoop
Ease of Use	Very easy and flexible; write simple scripts	More complex; requires C++ setup and JNI knowledge
Performance	Relatively lower (due to process overheads)	Higher (native performance through compiled code)
Development Speed	Fast prototyping	Suitable for performance-critical applications
Platform Independence	Platform-independent as long as the interpreter exists	Platform-dependent; requires native binaries
Typical Use Cases	Quick testing, rapid development with scripting	High-performance applications in C++